

Anti-patrones de Parcial + Estructura de Respuesta

Cómo responder (2 trade-offs + diagrama + componentes + escenarios) · anti-patrones que dan 2

Ingeniería de Software II · ITBA · 2026-1C · Metodología del parcial

RESUMEN

El apunte más rentable: cómo se aprueba y cómo se reprueba. **La estructura mínima aprobada = 2 trade-offs + 1 diagrama + componentes nombrados + escenarios cuantificados.** Los 5 anti-patrones que dan "2 automático" (cola síncrona, escalar sin LB, cachear mal, olvidar externos, OLAP para transaccional). Y un runbook de 7 pasos: el 80% de parciales prioriza Security / Availability / Scalability / Performance (source: cheat-sheet-estudio-pre-parcial.md, anti-patrones-parcial.md).

§1 Cómo responder — la estructura que aprueba

EN EL PARCIAL

La forma esperada (Clase 8): una respuesta de parcial **mínimamente aprobada** tiene exactamente cuatro cosas. Si falta alguna, está incompleta y suele aprobarse al límite o reprobar (source: clase-08-consultas-pre-parcial.md).

DEFINICIÓN

Estructura mínima aprobada:

1. **≥ 2 trade-offs** explícitos — uno atributo vs atributo (p. ej. Security vs Usability), uno costos vs tiempo (p. ej. on-prem ahorra opex pero retrasa el lanzamiento).
2. **1 diagrama** de componentes + conectores. El estilo no se prescribe; los componentes y flujos deben ser legibles.
3. **Componentes nombrados** — por cada caja: qué responsabilidad cubre y por qué está acá. Sin esto, el diagrama "es *hopes and dreams*".
4. **Escenarios cuantificados** — con métrica verificable, convención S-S-E-A-R-Measure del árbol de utilidad. Sin número, no es escenario.

*Estos cuatro no son adornos: son los **outputs canónicos del método ADD** (Attribute Driven Design). El parcial es la prueba de que internalizaste ese método (source: clase-08-consultas-pre-parcial.md).*

El método ADD aplicado al parcial — 6 pasos

1. **Leé el caso 2 veces.** Subrayá stakeholders, restricciones, volumen, regulación, criticidad.

2. **Identificá el top 4 de atributos.** En el 80% de los parciales son combinación de **SECURITY** **AVAILABILITY** **SCALABILITY** **PERFORMANCE** más uno de Interoperability / Fault Tolerance / Modifiability según el dominio.
3. **Ordenalos por prioridad** según el caso (Banco→Security #1; Streaming→Performance #1; IoT→Scalability #1; Salud→Reliability #1).
4. **Iterá por atributo:** para cada uno elegí mecanismo del bucket, dibujalo, escribí su escenario.
5. **Identificá 2 trade-offs** que surgen del diseño y resolvelos explícitamente.
6. **Revisá la lista de anti-patronos** (§4). Si caíste en uno, corregí *antes* de entregar.

(source: cheat-sheet-estudio-pre-parcial.md, §0 — método ADD aplicado al parcial.)

Estructura recomendada del escrito

1. Stakeholders y restricciones (5 líneas).
2. Top 4 atributos de calidad - priorizados y JUSTIFICADOS por el caso.
3. Escenarios cuantificados (1 por atributo top).
4. Diagrama de componentes (con sistemas externos).
5. Iteración por atributo: qué mecanismos elijo y por qué.
6. Trade-offs (mínimo 2).
7. Riesgos residuales + qué validaría con un prototipo.

Estructura derivada del esqueleto ATAM aplicado al Caso Compraventa de Acciones (source: cheat-sheet-estudio-pre-parcial.md).

REGLA DE ORO

La frase que la cátedra quiere oír (por cada iteración): "Prioricé $[A,B,C,D]$ porque el enunciado dice $[X]$. Para A elijo $[mecanismo]$ porque $[justificación]$, aceptando el trade-off $[T]$ que resuelvo con $[decisión]$. El riesgo residual es $[R]$, que validaría con $[prototipo / load test]$." (source: cheat-sheet-estudio-pre-parcial.md, §14).

§2 El número clave: escenarios cuantificados

CUÁNDO NO · CUIDADO

Mata la respuesta: escenarios con adjetivos — "rápido", "seguro", "escalable". Sin número, no es escenario, es un deseo.

EJEMPLO

Plantilla de escenario: "Bajo carga normal de N usuarios concurrentes (X req/s), el sistema responde al p95 en $\leq Y$ ms y al p99 en $\leq Z$ ms. Bajo pico de M usuarios, $p95 \leq Y'$ ms." Si el caso no da números, inventá razonables y **documentá la suposición** (source: cheat-sheet-estudio-pre-parcial.md, §12 paso 3).

§3 Anti-patrones que dan "2 automático"

EN EL PARCIAL

Por qué descalifican y no sólo restan puntos: revelan **incomprensión del modelo mental básico** del curso — cómo se asincroniza, cómo se escala, cómo se elige persistencia, cómo se integran sistemas externos. Rompen el método ADD (source: anti-patrones-parcial.md).

#1 — Cola como request-response síncrono

CUÁNDO NO • CUIDADO

Qué pasa: dibujar una cola (RabbitMQ, Kafka, SQS) entre dos servicios pero modelarla como "A pone el mensaje y **espera** la respuesta de B".

Por qué: la cola es por definición un **buffer asincrónico** que desacopla productor de consumidor. Si esperás la respuesta, no ganaste desacople y construiste algo más frágil (correlation IDs, reply queues, timeouts) que un HTTP request directo.

CUÁNDO SÍ • VENTAJAS

Qué hacer en su lugar: cola sólo para fire-and-forget, event-driven, work queues, ETL, retries. Si necesitás response asincrónico real → callback URL / webhook / async polling, no fingir RPC sobre cola. Si necesitás la respuesta para seguir → HTTP/gRPC síncrono (source: anti-patrones-parcial.md).

#2 — Escalamiento horizontal sin Load Balancer

CUÁNDO NO • CUIDADO

Qué pasa: "voy a escalar agregando réplicas" pero no aparece un LB en el diagrama.

Por qué: sin LB los N nodos no comparten carga — cada cliente sigue pegándole a uno solo. El LB es **parte constituyente** del escalado horizontal, no un accesorio.

CUÁNDO SÍ • VENTAJAS

Qué hacer en su lugar: dibujá el LB (DNS round-robin, Nginx/HAProxy, ALB/NLB, service mesh). **Excepciones legítimas a documentar:** sharding por clave (router determinístico), P2P/DHT, Anycast (LB a nivel red/BGP). Ojo: el LB es nuevo SPOF → multi-AZ o activo-pasivo con failover (source: anti-patrones-parcial.md, cheat-sheet-estudio-pre-parcial.md §2).

#3 — Cachear datos que no deberían cachearse (menos grave, igual penaliza)

CUÁNDO NO • CUIDADO

Qué pasa: cachear sin documentar invariantes, o cachear datos sensibles a freshness.

Por qué: el cache introduce acoplamiento temporal con la consistencia. Sin TTL ni invalidación, los stale reads son bugs no-deterministas y agujeros de seguridad (permisos cacheados tras una revocación).

CUÁNDO SÍ • VENTAJAS

Mínimo para cachear: (a) TTL o regla de invalidación; (b) política de coherencia (cache-aside, write-through, write-behind, refresh-ahead); (c) visibilidad de stale reads.

Casi nunca se cachean: saldos bancarios, ACLs/permisos, precios en subasta live, datos médicos críticos (source: anti-patrones-parcial.md).

#4 — Olvidarte los sistemas externos

CUÁNDO NO • CUIDADO

Qué pasa: el diagrama omite el procesador de pagos, el carrier, el IdP, el SMTP, el SDK del proveedor.

Por qué: cada integración externa tiene su propio modelo de fallas (timeout, throttling, partición, contrato cambiante) y su propio SLA. Olvidarlos esconde dependencias críticas que dominan el árbol de utilidad. En Healthcare.gov el verificador de identidad (Experian) fue un SPOF ignorado: cuando cayó, cayó todo el flow.

CUÁNDO SÍ • VENTAJAS

Qué representar: todo sistema externo del happy path con su contrato (REST/SOAP/gRPC), su modo de falla (sync timeout / async retry) y tu mitigación (circuit breaker / fallback / cola) (source: anti-patrones-parcial.md).

#5 — Usar bases OLAP para transaccional

CUÁNDO NO • CUIDADO

Qué pasa: proponer Snowflake/BigQuery/Redshift/columnares para guardar pedidos, usuarios, sesiones.

Por qué: OLAP es para queries analíticas de lectura sobre datos desnormalizados (latencia de segundos a minutos). El transaccional (OLTP) exige escritura concurrente, baja latencia (ms), ACID, normalización. OLAP para OLTP: 10× más caro, 100× más lento, sin ACID.

CUÁNDO SÍ • VENTAJAS

Patrón canónico: OLTP para operación + ETL/CDC + OLAP para analítica. **Nunca al revés** (source: anti-patrones-parcial.md).

Otros igual de graves (no enumerados pero penalizan)

- **Microservicios desde el día 0 con 5 ingenieros** — overhead operativo > beneficio.
- **Big bang release sin staging ni canary** — lección Healthcare.gov.
- **Dibujar el diagrama antes de articular los drivers** — viola el orden del método ADD.
- **Métrica = "rápido"/"seguro"/"escalable"** — sin número no es escenario.
- **Asumir conectividad perfecta** entre componentes distribuidos — viola las Falacias de Computación Distribuida (L. P. Deutsch).

(source: *anti-patrones-parcial.md*, *cheat-sheet-estudio-pre-parcial.md* §11.)

§4 Runbook de 7 pasos bajo presión

Cronómetro mental: ~20-25 min por caso (source: cheat-sheet-estudio-pre-parcial.md §12).

TABLA 1. Runbook del parcial — qué hacer en cada minuto

Paso	Tiempo	Qué hacés
1. Leer el caso (×2)	2 min	Subrayar dominio, usuarios, volumen, regulación, restricciones, sistemas externos, criticidad.
2. Identificar drivers	3 min	Listar atributos relevantes → priorizar top 4 , justificando cada uno con una línea del caso.
3. Escenarios cuantificados	3 min	Uno por atributo top, con números (p95/p99, req/s). Documentar suposiciones.
4. Diagrama	5-7 min	Usuarios + Frontend/CDN + Edge (WAF, LB) + Backend (N inst.) + Persistencia (OLTP + réplicas + caché + OLAP) + Cola + Sistemas externos + protocolos (HTTPS).
5. Iteración por atributo	5-7 min	"Para [atributo] introduzco [mecanismo]; resuelve [escenario] pero introduce [trade-off]."
6. Trade-offs explícitos	2 min	Mínimo 2, diciendo <i>cómo</i> los resolvés en este caso.
7. Revisión final	2 min	Barrer el checklist anti-patrones (abajo). Si todo pasa → entregar.

REGLA DE ORO

Checklist final (paso 7):

- ¿Top 4 atributos justificados por el caso (no genéricos)?
- ¿Escenarios con números, no adjetivos?
- ¿Hay 2 trade-offs (no 0)?
- ¿Hay **Load Balancer** si dije "escalo horizontal"?
- ¿Las colas son asincrónicas (fire-and-forget)?
- ¿La caché tiene TTL/política?
- ¿Aparecen los sistemas externos?
- ¿No estoy usando OLAP para operación?
- ¿Aparece HTTPS en todas las conexiones públicas?

(source: cheat-sheet-estudio-pre-parcial.md §12 paso 7).

Mapa dominio → top de atributos (el 80% de los parciales)

En el 80% de los parciales el top sale de:

SECURITY

AVAILABILITY

SCALABILITY

PERFORMANCE

+ uno de Interoperability / Fault Tolerance / Modifiability según dominio
(source: cheat-sheet-estudio-pre-parcial.md §0).

TABLA 2. Patrón de mapeo dominio → top 4 atributos

Dominio	Top 4 atributos
Banca / Fintech	Security, Availability, Consistency, Auditability
E-commerce	Scalability, Availability, Performance, Interoperability
Streaming / CDN	Performance, Scalability, Availability, Cost
Salud / Telemedicina	Security, Reliability, Privacy, Interoperability (HL7/FHIR)
IoT / Smart City	Scalability, Fault Tolerance, Performance, Availability
Gov / Voto	Security, Auditability, Verifiability, Availability
Logística	Availability, Scalability, Interoperability, Performance

(source: cheat-sheet-estudio-pre-parcial.md §12 paso 2.)

FUENTE

Las distinciones finas también caen: **Available ≠ Reliable** (un programa que reinicia cada hora en 1 s es available ~99.97% pero no reliable). Si el dominio tiene operaciones en curso largas (subasta, videoconsulta, transferencia), domina **Reliability**, no Availability (source: clase-08-consultas-pre-parcial.md). **Auditability**: los logs van en storage separado con control de acceso distinto (separation of duties), no como tabla en la base operacional.

REGLA DE ORO

Regla de oro final: antes de entregar, releé el diagrama y barré la lista de anti-patrones. Un parcial técnicamente brillante que cae en un solo "2 automático" reprueba igual. La revisión de 2 minutos del paso 7 es el ROI más alto de todo el examen.